UNICHAT

Guida completa al codice

Architettura · funzionalità · API · database
interfaccia · realtime · sicurezza · manutenzione

Edizione tecnica / 20 ottobre 2026
32 pagine

Indice dei capitoli

Come leggere questa guida 4

1. Introduzione: cosa si può fare con UniChat 5

2. Mappa del repository e tecnologie 7

3. Avvio, configurazione e ciclo di vita 8

4. Database: entità, relazioni e vincoli 9

5. Autenticazione e profili 10

6. Autorizzazione, ruoli e audit 11

7. Server, università, canali e inviti 12

8. Messaggi, conversazioni e preferiti 13

9. Allegati e file 14

10. Amici, blocchi e presenza 15

11. Socket.IO e sincronizzazione in tempo reale 16

12. Chiamate private: WebRTC 17

13. Canali vocali di gruppo: LiveKit 18

14. Minigiochi in tempo reale 19

15. Frontend: struttura, stato e rendering 20

16. Interfaccia, temi, responsive e accessibilità 21

17. Ricerca globale e pannelli amministrativi 22

18. Sicurezza: cosa protegge il codice e cosa richiede attenzione 23

19. API HTTP: repertorio per dominio 24

20. Flussi completi: dalla UI al database 25

21. Test, verifica e manutenzione 26

22. Limiti attuali e glossario 27

Appendice A. File da aprire per orientarsi rapidamente 28

Appendice B. Regole pratiche per chi estende il progetto 29

Appendice C. Indice analitico dei moduli frontend 30

Appendice D. Indice analitico dei moduli backend 31

Architettura, funzionalità, flussi, API e manutenzione

Edizione basata sul repository del 20 ottobre 2026. Documento tecnico in italiano.

Come leggere questa guida

Questa guida descrive il comportamento implementato nei file del progetto, non un prodotto ipotetico. I percorsi sono relativi alla radice UniChat-Project. I nomi delle rotte indicano il percorso completo sotto /api. Alcune funzioni sono disponibili solo agli utenti con il ruolo o l'appartenenza necessari. I servizi esterni, come PostgreSQL e LiveKit, devono essere configurati perché le rispettive funzioni siano operative.

Una lettura efficace segue tre livelli: il capitolo 1 mostra cosa può fare una persona nel sito; i capitoli 2–8 seguono il percorso tecnico di una richiesta; i capitoli successivi analizzano singoli sottosistemi, sicurezza, esercizio e test. Il codice resta la fonte definitiva se il progetto cambia dopo la data di questa edizione.

1. Introduzione: cosa si può fare con UniChat

UniChat è un'applicazione web di comunicazione per comunità universitarie. Un utente può creare un account con email, username, nome, password e università, oppure iscriversi tramite un invito a un server. Dopo il login può modificare nome e avatar, scegliere un tema e uscire revocando la sessione.

Nell'interfaccia principale può navigare tra la Home, i server universitari e le conversazioni private. La Home mostra il messaggio di benvenuto; nella sidebar si trovano accessi rapidi ad amici, richieste di amicizia, aggiunta amici, messaggi diretti, preferiti e impostazioni. Le conversazioni private già avvenute mostrano foto profilo e nickname dell'altra persona. I menu contestuali di server e canali si aprono con il tasto destro, con supporto anche per tastiera e pressione prolungata su dispositivi touch.

Un utente può cercare persone per username, inviare e gestire richieste di amicizia, rimuovere amici e bloccare o sbloccare utenti. La presenza degli amici viene aggiornata in tempo reale. Per chattare in privato, i due utenti devono essere amici e non deve esserci un blocco tra loro. La sidebar conserva l'accesso ai contatti con cui sono stati scambiati messaggi o effettuate chiamate, anche quando una conversazione non è quella attualmente aperta.

I server raccolgono canali testuali e vocali. Un utente può creare un server della propria università, scoprirne altri resi visibili, chiedere di entrare o utilizzare un invito. Il proprietario o un amministratore del server gestisce richieste di ingresso, inviti, visibilità e canali. Un server appena creato contiene i canali generale, materiali e studio-vocale. I permessi effettivi sono sempre controllati dal backend.

Nei canali testuali e nelle chat private si inviano messaggi di testo e allegati. Il testo è limitato dal server a 1.000 caratteri per messaggio. I file ammessi includono immagini, PDF, archivi ZIP, file di testo, audio e video nei formati dichiarati dall'API. L'utente può eliminare i propri messaggi; nel canale un moderatore locale può eliminare messaggi altrui. Un messaggio eliminato diventa una traccia senza contenuto né allegato. I messaggi possono essere salvati nei preferiti, e la ricerca globale interroga server, canali, utenti o amici a seconda dell'ambito scelto.

Tra amici si possono avviare chiamate private audio e video. Il backend coordina richiesta, risposta, segnali di connessione e conclusione; il video e l'audio della chiamata privata passano tramite WebRTC nel browser. La cronologia delle chiamate private è registrata nel database. Nei canali vocali di gruppo, invece, LiveKit gestisce la stanza media: prima di entrare compare l'anteprima della propria webcam, con controlli di microfono e videocamera e il pulsante «Entra». Una volta connessi, compaiono i riquadri video degli altri partecipanti, l'indicazione di chi parla e i comandi per condividere schermo o finestra. La trascrizione live non è presente nell'interfaccia attuale.

UniChat include due minigiochi in tempo reale: tris e impiccato. Si può invitare un amico o condividere un link di gioco nel contesto consentito. Lo stato del gioco è distribuito tramite Socket.IO; è conservato in memoria del processo e quindi non sopravvive al riavvio.

Le persone con privilegi appropriati vedono pannelli amministrativi. Esistono funzioni per consultare utenti e server, cambiare ruoli, sospendere o riattivare account, gestire server e canali, nominare

amministratori locali e consultare un registro delle azioni amministrative. Le funzioni del Creator/SuperAdmin comprendono la gestione delle università e alcune operazioni riservate; il controllo è applicato sia nell'interfaccia sia, soprattutto, nelle API.

L'interfaccia è organizzata come un'app a tutta finestra: barra delle sezioni, sidebar, area centrale e, quando serve, pannello dei membri. Il font principale è Roboto. Si possono scegliere i temi chiaro, scuro e «Nero e Rame»; la scelta è memorizzata in localStorage e applicata all'avvio prima del caricamento degli stili. Su schermi piccoli la navigazione laterale si apre come pannello mobile.

2. Mappa del repository e tecnologie

frontend/index.html contiene la struttura delle schermate, dei dialoghi e dei controlli. I file frontend/assets/css/styles.css, redesign.css, theme.css e app-shell.css definiscono componenti, varianti cromatiche e layout. frontend/assets/js/app.js compone l'applicazione browser importando moduli ES. Non è presente un framework SPA: lo stato è un oggetto JavaScript, gli elementi sono referenziati dal DOM e le viste sono aggiornate dai controller.

backend/src/main.js avvia un unico server HTTP con Express e Socket.IO. backend/src/app.js configura middleware, file statici e router. backend/src/routes definisce gli indirizzi HTTP; backend/src/controllers applica le regole funzionali; backend/src/middleware applica controlli trasversali; backend/src/sockets gestisce eventi in tempo reale; backend/src/utils raccoglie operazioni su file, inviti e sessioni; backend/src/lib centralizza Prisma e l'autorizzazione.

backend/prisma/schema.prisma è il modello dati. Le cartelle backend/prisma/migrations documentano l'evoluzione dello schema SQL. backend/test contiene test automatici Node. docker-compose.yml avvia backend e PostgreSQL in container; backend/Dockerfile prepara l'immagine applicativa. README.md fornisce istruzioni sintetiche: questa guida aggiunge il percorso tecnico dettagliato.

La catena di una normale azione è: browser → evento DOM → controller frontend → api() o Socket.IO → autenticazione/autorizzazione backend → Prisma/PostgreSQL → risposta HTTP o evento realtime → aggiornamento dello stato e del DOM. I file multimediali di una stanza LiveKit prendono un percorso separato: browser → token breve rilasciato dal backend → LiveKit. Le chiamate private usano Socket.IO per il signaling e WebRTC per i media.

3. Avvio, configurazione e ciclo di vita

backend/package.json espone npm run dev con Nodemon, npm start, npm test, npm run check e comandi Prisma. Per lo sviluppo locale è necessario che PostgreSQL sia raggiungibile con l'URL indicato da DATABASE_URL. In Docker Compose, PostgreSQL ascolta nella rete interna come postgres:5432 e viene esposto all'host sulla porta 5433. Il backend si presenta sulla porta 5000. Il controllo di salute del database precede l'avvio del container backend.

backend/src/config/env.js carica backend/.env e poi backend/.env.local, che può sovrascrivere le variabili precedenti. I valori reali di password, pepper e credenziali LiveKit non devono essere pubblicati. Il modulo valida limiti di upload, numero di tentativi di login, durata delle chiamate, indirizzi STUN/TURN e URL LiveKit. In produzione richiede un SESSION_TOKEN_PEPPER robusto. I valori configurabili comprendono PORT, CORS_ORIGIN, DATABASE_URL, SESSION_DURATION_DAYS, SALT_ROUNDS, MAX_ATTACHMENT_BYTES, MAX_ATTACHMENT_STORAGE_BYTES, WEBRTC_STUN_URLS, WEBRTC_TURN_URLS, WEBRTC_TURN_SHARED_SECRET, LIVEKIT_URL, LIVEKIT_API_KEY e LIVEKIT_API_SECRET.

main.js crea l'app Express, la incapsula in un server HTTP, collega Socket.IO e rende io disponibile ai controller tramite app.set('io', io). All'avvio marca come concluse le chiamate private rimaste negli stati RINGING o ACCEPTED dopo un riavvio: non esiste più una connessione media valida per quelle chiamate. Avvia inoltre una pulizia periodica degli allegati orfani. serverShutdown.js coordina la chiusura del server, delle connessioni Socket.IO e del client Prisma quando arrivano SIGINT o SIGTERM.

app.js imposta intestazioni di sicurezza di base, CORS, parsing JSON e distribuzione statica del frontend. Espone il bundle browser di LiveKit sotto /assets/vendor/livekit-client.js. Gli avatar pubblici sono serviti da /uploads/:filename solo se un utente del database usa quel percorso come avatar e l'estensione è un'immagine ammessa. Gli allegati dei messaggi seguono invece le rotte autenticate /api/uploads. Per le API, il router aggregatore si trova in backend/src/routes/index.js; notFoundHandler ed errorHandler concludono la catena.

Se npm run dev stampa PrismaClientInitializationError e «Can't reach database server at 127.0.0.1:5433», Express è partito ma PostgreSQL non è raggiungibile all'indirizzo configurato. È necessario avviare il database o correggere DATABASE_URL, poi applicare le migrazioni Prisma. Se una stanza vocale restituisce «LiveKit non configurato», almeno una delle tre variabili LiveKit non è caricata nel processo backend. La connessione WebRTC richiede inoltre HTTPS o localhost per l'accesso a webcam e microfono.

4. Database: entità, relazioni e vincoli

Prisma usa PostgreSQL. University contiene nome e dominio univoci; collega utenti e server. User contiene email e username univoci, hash della password, nome, avatar, ruolo, università opzionale, eventuale sospensione e dati per autenticazione a più fattori. I ruoli del modello sono STUDENT, PROFESSOR, ADMIN, PLATFORM_ADMIN e SUPERADMIN. La presenza del ruolo nel database non basta da sola per tutte le autorizzazioni: i controlli di authorization.js considerano anche stato dell'utente, appartenenza al server e vincoli del Creator.

Session contiene solo l'hash del token, utente proprietario, scadenza e campi relativi a secondo fattore. Il token in chiaro viene restituito al browser durante il login, ma non è salvato in chiaro nella tabella. MfaRecoveryCode conserva hash dei codici di recupero. Lo schema include campi TOTP come segreto cifrato, attivazione, tentativi falliti e lock temporaneo. I percorsi HTTP correnti di authRoutes.js espongono registrazione, login, logout, lettura e aggiornamento profilo; non espongono un flusso TOTP completo. Questo è un punto importante quando si distingue ciò che lo schema può memorizzare da ciò che la UI può usare oggi.

Server appartiene a un'università e a un proprietario. Contiene descrizione, codice invito, scadenza, numero massimo di utilizzi, contatore, revoca e visibilità nella scoperta. ServerMember implementa la relazione molti-a-molti fra utenti e server e registra isAdmin. La coppia utente–server è univoca. ServerJoinRequest memorizza richieste PENDING, ACCEPTED o DECLINED, con vincolo di unicità per coppia.

Channel appartiene a un server ed è TEXT o VOICE. Message appartiene a un canale e a un autore. PrivateMessage collega mittente e destinatario. Entrambi contengono testo, metadati di un eventuale allegato, data di creazione e campi deletedAt/deletedById per la cancellazione logica. Le tabelle ChannelMessageFavorite e PrivateMessageFavorite associano un messaggio a un utente che lo ha salvato, usando chiavi composte per impedire duplicati.

Friendship rappresenta una relazione fra due utenti, con stato PENDING o ACCEPTED, autore effettivo della richiesta e vincolo sulla coppia. BlockedUser registra chi blocca chi, con coppia univoca. PrivateCall registra identificativo, chiamante, destinatario, stato e orari di inizio, risposta e termine. AdminAuditLog registra autore, azione, tipo e ID della risorsa, server eventuale, esito e data. Gli indici su ruoli, membri, conversazioni e log velocizzano i filtri principali.

Le migrazioni in backend/prisma/migrations devono essere applicate nel loro ordine. Modificare soltanto schema.prisma non aggiorna una banca dati già esistente. Le relazioni con onDelete: Cascade sono esplicite per varie entità dipendenti; altre operazioni di eliminazione richiedono logica applicativa. Il database conserva messaggi, amicizie e cronologia delle chiamate, mentre stato di presenza, sessioni WebRTC e giochi in corso vivono anche in memoria o nelle connessioni attive.

5. Autenticazione e profili

authRoutes.js instrada POST /api/auth/register, POST /api/auth/login, POST /api/auth/logout, GET /api/auth/me, PATCH /api/auth/me e l'alias GET /api/auth/authentication. Il limite per tentativi di autenticazione in authRateLimit.js conta le richieste per indirizzo IP in una finestra di 15 minuti; in caso di superamento risponde 429 con Retry-After. Questo contatore è in memoria del singolo processo.

La registrazione in authControllers.js normalizza email e username, valida la password tra 12 e 128 caratteri, verifica unicità di email e username e richiede un'università oppure un invito valido. Il codice riserva l'identità del Creator e impedisce una registrazione ordinaria con email o username riservati. La password viene cifrata con bcrypt secondo SALT_ROUNDS. La creazione dell'utente e l'eventuale riscatto dell'invito avvengono in transazione, così un errore non lascia un'iscrizione incompleta.

Al login, il server confronta la password con l'hash bcrypt, rifiuta gli account sospesi, genera 32 byte casuali e invia al browser un token esadecimale. sessionTokens.js produce un hash HMAC-SHA256 del token con un pepper del server; nella tabella Session viene memorizzato l'hash. authMiddleware.js richiede Authorization: Bearer <token>, ricostruisce l'hash, cerca la sessione, controlla scadenza e sospensione, poi assegna req.user e req.session. Le rotte protette ereditano questo controllo.

Il browser conserva il token in localStorage con chiave unichat:token; api.js lo aggiunge automaticamente alle chiamate HTTP. bootstrapData.js prova a recuperare la sessione e poi carica i dati dell'area di lavoro. Il logout elimina la sessione sul server, disconnette i socket che usano quel token, pulisce stato e token locale e interrompe voce o chiamata in corso. L'aggiornamento del profilo accetta nome e avatar, anche tramite multipart. La risposta dei profili seleziona campi pubblici appropriati e non include la password.

6. Autorizzazione, ruoli e audit

backend/src/lib/authorization.js concentra le regole. isActive esclude account sospesi. isSuperadmin richiede ruolo SUPERADMIN e corrispondenza dell'email riservata al proprietario del sito. canModerateGlobal ammette ruoli globali definiti dal codice o il SuperAdmin. getServerScope legge proprietà e appartenenza; canManageServer abilita proprietario, admin locale o SuperAdmin; canModerateServerMessages riguarda proprietario e admin locale. I middleware requireGlobalModerator, requireSuperadmin e requireServerManager producono 403 quando il permesso manca.

adminRoutes.js applica autenticazione e controllo di ambito. GET /api/admin/overview raccoglie informazioni paginabili per i pannelli; GET /api/admin/audit legge il registro. Le rotte di aggiornamento riguardano server, canali, ruoli, sospensioni e incarichi di admin locale. La rimozione di un utente è riservata al SuperAdmin. roleControllers.js limita i ruoli assegnabili a STUDENT, PROFESSOR e PLATFORM_ADMIN: il ruolo SuperAdmin non viene assegnato da una normale chiamata API. adminControllers.js implementa le modifiche amministrative e aggiorna o disconnette i client dove richiesto.

adminAudit.js collega un listener all'evento finish della risposta HTTP. Registra l'azione, la risorsa, l'attore e un esito SUCCESS, DENIED o ERROR. Questa registrazione è asincrona: l'errore di scrittura viene registrato nel log server, mentre la risposta HTTP è già conclusa. La UI amministrativa di frontend/assets/js/modules/controllers/admin.js mostra le azioni disponibili in base alle capability restituite dal backend; nascondere un pulsante non sostituisce mai il controllo del server.

7. Server, università, canali e inviti

universityRoutes.js espone la lista delle università a chi deve registrarsi. Creazione e cancellazione richiedono il SuperAdmin. serverRoutes.js fornisce elenco server dell'utente, scoperta dei server visibili, creazione, dettaglio, ingresso, richieste e gestione di inviti e canali. Il Creator può selezionare l'università al momento di creare un server; per gli altri utenti il server usa l'università del proprio account.

La creazione di un server in serverControllers.js genera un codice invito casuale, crea contemporaneamente proprietario come membro e tre canali predefiniti. La scoperta mostra server visibili dell'università dell'utente. Premere «Entra» o usare un invito produce una richiesta in attesa quando l'utente non è già membro: la richiesta deve essere accettata da chi gestisce il server. Le funzioni di invito in serverInvites.js validano codice, scadenza, revoca e numero massimo di usi; il codice può essere ruotato o revocato.

I canali sono di tipo testuale o vocale. La loro creazione richiede un gestore del server; il nome è normalizzato. Un membro può aprire i canali e leggere quelli consentiti. messageControllers.findAccessibleChannel verifica sia esistenza del canale sia appartenenza al server: lo stesso controllo viene riutilizzato dalle API dei messaggi e dal rilascio di token vocali. Lasciare un server rimuove la membership e disconnette i socket dell'utente; il proprietario, invece, elimina il server. I menu contestuali della UI permettono di rinominare o eliminare canali e di aprire impostazioni di server quando i permessi lo consentono.

8. Messaggi, conversazioni e preferiti

Per i canali, GET /api/channels/:channelId/messages restituisce fino a 80 messaggi ordinati per data crescente; POST crea un messaggio e DELETE /:messageId lo elimina logicamente. Le chat private usano GET /api/private/:userId/messages, POST e DELETE equivalenti, più GET /api/private/:userId/calls per la cronologia. Le rotte HTTP di creazione accettano un corpo multipart quando è presente un allegato. Per i messaggi di testo in canale, il browser usa anche l'evento Socket.IO send_message.

messageControllers.js centralizza i campi restituiti al client, riduce il testo a 1.000 caratteri e impedisce i messaggi vuoti senza allegato. Per i canali impone il tipo TEXT; per i privati verifica che il destinatario esista e che i due utenti siano amici accettati e non bloccati. Dopo una scrittura HTTP, emette message_created nel canale o private_message_created nelle stanze personali. L'evento socket send_message esegue a sua volta autenticazione e verifica di accesso prima di scrivere in Prisma e notificare il canale.

La lista GET /api/private/conversations non usa una tabella separata delle conversazioni: raggruppa messaggi inviati, ricevuti e chiamate in uscita o entrata, prende l'attività più recente per ciascun interlocutore e ordina l'elenco. Il frontend mostra avatar, nickname e ultima attività. Questo spiega perché una chiamata privata può creare o mantenere un contatto nella lista anche senza un nuovo messaggio.

La cancellazione usa messageDeletion.js: il record resta presente con deletedAt e deletedById, mentre testo e metadati dell'allegato diventano vuoti. La funzione evita una seconda cancellazione concorrente, prova a rimuovere il file se non è più referenziato e invia message_deleted ai client. I preferiti sono separati per messaggi di canale e messaggi privati. GET /api/favorites aggrega i due tipi, PUT /api/favorites/:kind/:messageId salva e DELETE rimuove; il controller verifica che l'utente abbia accesso al messaggio.

messageViews.js costruisce i nodi DOM del flusso, i comandi dei preferiti, le tracce dei messaggi cancellati e la cronologia delle chiamate. messageAttachments.js valida file prima dell'invio, compone FormData, mostra anteprime e libera gli URL blob: temporanei. navigation.js coordina selezione di server, canale e chat privata; channels.js aggiorna i canali dopo creazione, rinomina o eliminazione.

9. Allegati e file

Gli allegati ai messaggi hanno un limite predefinito di 25 MiB e al massimo un file per messaggio. /api/uploads/config restituisce limiti e formati correnti al browser. uploadParser.js usa Multer per ricevere multipart. Per i messaggi, messageUpload.js e attachmentTypes.js controllano estensione, contenuto e firma dei principali formati; i file testuali vengono decodificati come UTF-8 e controllati contro byte di controllo non ammessi. La quota totale è regolata da MAX_ATTACHMENT_STORAGE_BYTES.

Gli allegati privati usano nomi interni casuali in private_uploads. privateAttachments.js calcola lo spazio occupato e rimuove file orfani più vecchi di un'ora quando nessun messaggio li referenzia. Il cleanup si avvia sia all'avvio del server sia periodicamente. Gli avatar seguono una cartella diversa e un percorso pubblico condizionato alla presenza di un riferimento nel database.

uploadRoutes.js richiede autenticazione per ogni download di allegato. attachmentAccess.js verifica che il richiedente appartenga al server del messaggio oppure sia coinvolto nella conversazione privata ancora accessibile. Il download è restituito come application/octet-stream con Content-Disposition: attachment, nome ripulito, nosniff, cache privata disabilitata e CSP sandbox. Il percorso del file è convalidato prima dell'accesso, così un nome contenente separatori non può trasformarsi in un percorso arbitrario.

10. Amici, blocchi e presenza

friendRoutes.js espone elenco amici, ricerca, richieste ricevute/inviate, invio, accettazione, rifiuto, cancellazione, rimozione, elenco bloccati, blocco e sblocco. friendControllers.js normalizza gli username, conserva la relazione evitando coppie duplicate e distingue il richiedente dal destinatario anche quando i due ID sono canonizzati per unicità. Bloccare una persona impedisce i normali contatti privati; i controlli server sono ripetuti per ogni invio o chiamata.

Il controller pubblica gli eventi di aggiornamento nelle stanze personali user:<id> di Socket.IO. presence.js calcola gli amici che possono vedere la presenza, controlla se una stanza personale ha socket connessi, invia uno snapshot quando ci si autentica e notifica i cambiamenti online/offline. La presenza indica connessione corrente, non una disponibilità storica nel database. friends.js aggiorna elenchi, contatori e richieste; settings.js mostra blocchi e comandi per la gestione dei contatti.

11. Socket.IO e sincronizzazione in tempo reale

sockets/index.js registra i gestori di chat, voce, chiamate e giochi su ogni connessione. Il client invia authenticate con il token di sessione; sockets/shared.js ricostruisce l'utente tramite lo stesso hash della sessione HTTP e controlla scadenza e sospensione. Il socket autenticato entra nella stanza personale. L'evento disconnect aggiorna la presenza e lascia pulire gli stati temporanei. I gestori di operazioni sensibili ricontrollano il token o la sessione, così conoscere l'ID di una stanza non dà automaticamente accesso ai dati.

chatHandlers.js gestisce join_channel e send_message. Il server consente l'adesione alla stanza di canale solo a un membro del server, e notifica message_created dopo il salvataggio. I controller HTTP emettono eventi equivalenti per le scritture effettuate tramite API. voiceHandlers.js mantiene l'appartenenza logica alle stanze vocali e gli eventi di entrata/uscita; il media multipartecipante è gestito da LiveKit, non da Socket.IO. Non esiste più un gestore di trascrizione live nel percorso corrente.

socket.js lato browser collega gli eventi ricevuti agli aggiornamenti della UI: messaggi nuovi o cancellati, presenza, amici, chiamate e giochi. La separazione tra API e realtime è pratica: HTTP esegue operazioni richiedibili e restituibili; Socket.IO comunica cambiamenti e coordina scambi che richiedono bassa latenza. Il codice presume un singolo processo Socket.IO per registri in memoria, chiamate e giochi: una distribuzione su più istanze richiederebbe un adattatore condiviso e una strategia esplicita per lo stato temporaneo.

12. Chiamate private: WebRTC

callHandlers.js riceve private_video_call_request, verifica che i due utenti possano contattarsi, crea il record PrivateCall, avvisa il destinatario e tiene un registro temporaneo della chiamata. private_video_call_answer accetta o rifiuta; private_video_call_signal passa offer, answer e candidate ICE al socket dell'altro partecipante dopo aver validato struttura e appartenenza; private_video_call_end conclude la chiamata. La disconnessione di un partecipante termina le chiamate interessate. Timeout di squillo e durata massima sono configurabili in env.js.

calls.js lato browser chiede permessi per microfono e, nelle videochiamate, webcam; costruisce RTCPeerConnection, aggiunge le tracce locali, negozia offer/answer e raccoglie candidate ICE. GET /api/rtc/config fornisce server STUN e, se configurati, credenziali TURN temporanee generate sul server. I segnali viaggiano via Socket.IO; audio e video viaggiano tramite WebRTC. I pulsanti di microfono, videocamera, risposta, rifiuto e chiusura agiscono sullo stato locale e sugli eventi di chiamata. Senza un TURN raggiungibile alcune coppie di reti possono non stabilire il collegamento.

La cronologia deriva dai record PrivateCall; l'utente può visualizzarla per contatto. Il riavvio del backend chiude logicamente le chiamate che risultavano aperte. Questa è una scelta coerente con il registro delle chiamate e dei socket tenuto in memoria.

13. Canali vocali di gruppo: LiveKit

POST /api/rtc/voice-token richiede una sessione valida e un channelId di un canale VOICE accessibile all'utente. Se l'infrastruttura LiveKit non è configurata, risponde 503 con il messaggio «LiveKit non configurato». Se è configurata, crea un token LiveKit di 15 minuti per la stanza unichat-voice-<id canale>, con permessi di ingresso, pubblicazione e sottoscrizione. Ogni collegamento riceve un'identità unica derivata dall'utente e da un UUID. Le credenziali API segrete rimangono nel backend; il browser riceve soltanto URL e token a durata limitata.

voiceRtc.js prepara la vista locale prima dell'ingresso. Usa createLocalTracks per ottenere anteprima webcam e microfono nel contesto sicuro del browser. Il controllo di videocamera funziona prima dell'ingresso e dopo la connessione; il microfono segue mute e deafen dell'area account. Premendo «Entra» il browser richiede il token, si collega a LiveKit, pubblica le tracce attive e aggiorna i controlli. Errori di permesso dei dispositivi, di contesto HTTPS o di connessione producono uno stato leggibile nel pannello.

I partecipanti sono rappresentati da riquadri video. Il codice aggiunge o rimuove tile in base alle tracce disponibili, mostra un segnaposto se la camera è spenta e evidenzia chi sta parlando in base agli eventi di active speakers di LiveKit. La condivisione dello schermo o di una finestra genera una tile dedicata e può essere interrotta dal relativo comando. voicePanel.js decide quali elementi del pannello mostrare in base al canale selezionato e allo stato di connessione. Uscire ferma le tracce e disconnette la stanza; può riaprire l'anteprima nello stesso canale.

14. Minigiochi in tempo reale

gameHandlers.js supporta tris (TICTACTOE) e impiccato (HANGMAN). Le richieste private sono consentite tra amici accettati; un collegamento può essere condiviso nel contesto di un server. Gli eventi includono richiesta, accettazione, mossa, riavvio, creazione link e ingresso tramite link. Il server calcola risultati, turno corrente e stato da inviare ai giocatori, impedendo che un client decida da solo l'esito.

gameLauncher.js apre il selettore, sceglie avversario e tipo di partita, invia inviti o link e gestisce le risposte. gameBoard.js disegna tabellone, lettere dell'impiccato, stato del turno e risultato. gameButtonControllers.js crea piccoli controller animati attorno al pulsante gioco, rispettando la preferenza di movimento ridotto. La mappa games nel backend è in memoria: in caso di restart le partite in corso sono perse e non vanno interpretate come dati persistenti.

15. Frontend: struttura, stato e rendering

index.html definisce shell, barra principale, sidebar, header, contenuto, composer, pannello dei membri, area vocale, area chiamata e dialoghi. dom.js centralizza gli elementi con querySelector: ciò evita ricerche ripetute in molti controller, ma rende essenziale mantenere coerenti gli ID quando si modifica l'HTML. state.js conserva token, utente, server, canale, destinatario privato, amici, richieste, preferiti, presenza, stato gioco, socket, voce e chiamata. Gli ID attivi determinano la schermata visibile e l'azione del composer.

app.js crea un contesto con createAppContext(), registra i controller in ordine e chiama app.start(). Ogni controller aggiunge metodi a app con Object.assign, condividendo funzioni senza importare direttamente tutti gli altri moduli. startup.js avvia splash, layout, listener DOM e bootstrap asincrono. bootstrapData.js carica università e dati dell'area di lavoro, controllando la sessione esistente. api.js è il punto unico per fetch, intestazione Bearer, serializzazione JSON, gestione di FormData e messaggi di errore.

events.js collega i form e i pulsanti a funzioni del contesto: login, registrazione, logout, creazione server/canale, inviti, ricerca amici, invio messaggi, allegati, profilo, impostazioni, giochi, chiamate e voce. La logica di visualizzazione è distribuita nei controller: shell.js gestisce Home, dock account e overview; shellLayout.js gestisce pannelli, menu contestuali, ridimensionamento sidebar, drawer mobile e lista dei messaggi diretti; navigation.js sceglie contenuto; serverLists.js visualizza server e richieste; friends.js visualizza amicizie; globalSearch.js governa la ricerca; messageViews.js e messageAttachments.js governano conversazioni e file; admin.js e settings.js governano i pannelli dedicati.

core.js fornisce accessori per server, canale e interlocutore attivi e funzioni sui codici invito. ui.js contiene componenti DOM riutilizzabili come avatar, badge, conferme e toast. attachments.js contiene dimensioni e riconoscimento dei tipi usati dal browser. Il rendering usa proprietà testuali del DOM per i dati utente dove possibile; qualunque nuova funzione che costruisca HTML deve continuare a trattare contenuto e nomi ricevuti dal server come dati non fidati.

16. Interfaccia, temi, responsive e accessibilità

La shell usa quattro file CSS in cascata. styles.css definisce la base originale; redesign.css riorganizza schermate e componenti; theme.css definisce variabili e adattamenti per i temi; app-shell.css rifinisce barra laterale, pannelli, proporzioni e palette «Nero e Rame». Le variabili CSS coprono superfici, testo, bordi, accent color e stati. Il tema rame usa sfondo generale vicino a #090909, rail #0d0d0d, sidebar #121212, contenuto #181818, superfici rialzate #202020 e accento #cc5500. Il font preferito è Roboto, con Arial e sans-serif come ripiego.

Lo script iniziale nell'head di index.html legge unichat-theme da localStorage e assegna data-theme alla radice prima del caricamento dei fogli di stile: questo riduce il lampeggio del tema sbagliato. settings.js aggiorna immediatamente il tema selezionato. I temi disponibili nella UI sono chiaro, scuro e Nero e Rame. La preferenza del tema è locale al browser, non una colonna dell'account nel database; il progetto corrente non presenta una modalità automatica «segui sistema» nella scelta delle anteprime.

app-shell.css adatta la vista a larghezze ridotte e contiene una regola prefers-reduced-motion. Il drawer mobile conserva una via per riaprire la navigazione. Dialoghi HTML, etichette accessibili, pulsanti focusable, stati focus-visible, tasto destro, ContextMenu/Shift+F10 e pressione touch offrono modalità d'uso diverse. Quando si modifica una finestra o un menu, vanno controllati apertura, chiusura, focus e ritorno al pulsante attivante.

17. Ricerca globale e pannelli amministrativi

GET /api/search è una rotta autenticata. searchControllers.js accetta ambiti explore, member, channels, users e friends; limita e normalizza la query, interroga con Prisma le entità appropriate e restituisce risultati con paginazione. globalSearch.js apre il dialogo dalla barra superiore, ritarda leggermente le richieste durante la digitazione e visualizza risultati e pagine. La ricerca della sidebar è stata rimossa: l'accesso visibile è il controllo di ricerca globale nell'header.

I pannelli amministrativi hanno viste distinte per amministrazione e SuperAdmin. admin.js chiede overview e audit, rende righe e azioni e invoca le rotte protette. La sezione impostazioni dell'utente resta separata: profilo, aspetto, amicizie bloccate e comandi account. L'area account in basso nella sidebar mostra nome e ruolo; impostazioni e amministrazione sono accessibili da pulsanti dedicati secondo il ruolo. L'utente non acquisisce permessi solo perché una voce è visibile: ogni richiesta passa per l'autorizzazione backend.

18. Sicurezza: cosa protegge il codice e cosa richiede attenzione

La sicurezza si basa su più livelli: validazione dei dati in controller, sessione verificata a ogni richiesta protetta, controllo di appartenenza al server, permessi per ruoli e risorse, verifica di amicizia/blocco per interazioni private, download di allegati autorizzato, limiti su file e rate limit sul login. Express disabilita X-Powered-By e invia nosniff, Referrer-Policy e X-Frame-Options. Gli allegati sono serviti come download, con CSP sandbox e cache privata disabilitata.

I token di sessione sono in localStorage: uno script malevolo eseguito nel contesto del sito potrebbe leggerli. Perciò è importante non inserire HTML non affidabile nel DOM, usare HTTPS in produzione, aggiornare dipendenze e considerare una Content Security Policy globale. La CSP sandbox sugli allegati non equivale a una CSP globale per l'app. Il rate limit e alcuni registri realtime sono in memoria: più processi o restart ne modificano il comportamento. Le credenziali di LiveKit, il pepper e la password del database devono restare nei file locali o nel gestore di segreti dell'ambiente, mai nel frontend o nel PDF.

Un'ulteriore attenzione riguarda la coerenza del rilascio: la scelta del tema è per browser; giochi e chiamate attive non sopravvivono al riavvio; l'accesso al microfono dipende da HTTPS o localhost; la qualità delle chiamate private dipende dalla configurazione ICE/TURN e dalla rete. Questi sono limiti osservabili nell'implementazione, non errori del modello dati.

19. API HTTP: repertorio per dominio

Autenticazione: POST /api/auth/register crea l'account; POST /api/auth/login restituisce token e profilo; POST /api/auth/logout revoca la sessione; GET /api/auth/me e GET /api/auth/authentication leggono l'utente; PATCH /api/auth/me aggiorna nome/avatar.

Università: GET /api/universities elenca; POST /api/universities e DELETE /api/universities/:id richiedono SuperAdmin. L'alias /api/university usa lo stesso router.

Server: GET /api/servers elenca quelli dell'utente; POST /api/servers crea; GET /api/servers/available scopre; GET /api/servers/join-requests legge richieste gestibili; POST /api/servers/join-by-invite usa un codice; POST /api/servers/join-requests/:requestId/accept e DELETE /api/servers/join-requests/:requestId rispondono alle richieste; GET /api/servers/:id legge il dettaglio; POST /api/servers/:id/join invia una richiesta; DELETE /api/servers/:id elimina o lascia; POST /api/servers/:id/channels crea un canale. GET, POST e DELETE /api/servers/:id/invite leggono, ruotano o revocano l'invito; PATCH /api/servers/:id/discoverability cambia visibilità; DELETE /api/servers/:id/messages/:messageId modera un messaggio.

Canali e privati: GET/POST /api/channels/:channelId/messages e DELETE /api/channels/:channelId/messages/:messageId; GET /api/private/conversations; GET/POST /api/private/:userId/messages; DELETE /api/private/:userId/messages/:messageId; GET /api/private/:userId/calls.

Amici: GET /api/friends, /api/friends/blocked, /api/friends/requests e /api/friends/search; POST /api/friends invia richiesta; POST /api/friends/requests/:id/accept accetta; DELETE /api/friends/requests/:id rifiuta; DELETE /api/friends/requests/:id/cancel annulla; DELETE /api/friends/:userId rimuove; POST /api/friends/:userId/block blocca; DELETE /api/friends/blocked/:userId sblocca.

Ricerca, file, preferiti e media: GET /api/search; GET /api/uploads/config; GET /api/uploads/:filename e /api/uploads/private-uploads/:filename scaricano file autorizzati; GET /api/favorites, PUT/DELETE /api/favorites/:kind/:messageId; GET /api/rtc/config fornisce ICE; POST /api/rtc/voice-token fornisce token LiveKit. GET /api/health restituisce {status: "ok"} e non verifica da solo lo stato del database.

Admin: GET /api/admin/overview e /audit; PATCH /api/admin/servers/:id; PATCH/DELETE /api/admin/servers/:serverId/channels/:channelId; DELETE /api/admin/servers/:id; POST/DELETE /api/admin/users/:id/suspension; PATCH /api/admin/users/:id/role; PUT/DELETE /api/admin/servers/:serverId/admins/:userId; DELETE /api/admin/users/:id. Ogni rotta ha limiti di ruolo e ambito specifici.

20. Flussi completi: dalla UI al database

Registrazione: il form in index.html viene ascoltato da events.js; il browser crea un FormData, aggiunge l'eventuale codice invito e invia /auth/register tramite api.js. La rotta applica rate limit e parser multipart. Il controller valida campi, carica università o invito, calcola hash bcrypt e crea l'utente in transazione. Il frontend mostra l'esito e passa al form di login.

Messaggio testuale di canale: il composer produce testo in events.js; se non c'è file invia send_message sul socket. chatHandlers.js autentica, verifica il canale, crea il record Prisma e pubblica message_created nella stanza channel:<id>. Ogni client in quella stanza riceve l'evento; socket.js chiama il rendering in messageViews.js. Se c'è un file, il browser usa HTTP multipart e il controller emette un evento equivalente dopo il salvataggio.

Messaggio privato: la selezione del contatto imposta activePrivateUserId e activeConversationType. La richiesta POST /api/private/:userId/messages verifica amicizia e assenza di blocchi, salva testo/allegato e pubblica private_message_created nelle due stanze personali. La lista conversazioni viene ricostruita dalla cronologia dei messaggi e delle chiamate; il frontend aggiorna avatar e nickname in sidebar.

Ingresso in canale vocale: l'utente seleziona il canale; voicePanel.js mostra il pannello e voiceRtc.js avvia l'anteprima locale. Dopo la scelta di microfono e camera, il clic su «Entra» richiede /rtc/voice-token. Il backend verifica la membership, firma un token LiveKit breve e restituisce URL/token. Il browser si collega alla stanza, pubblica le tracce e ascolta le tracce dei partecipanti; le tile e lo stato speaking si aggiornano al variare della stanza.

Richiesta di amicizia: la UI chiama /friends; il controller verifica utenti e blocchi, crea o aggiorna la relazione con stato PENDING e notifica gli interessati. L'accettazione porta lo stato ad ACCEPTED; i due diventano eleggibili per chat, chiamate e giochi privati. Se una persona viene bloccata, i controlli canContact impediscono nuovi scambi privati.

21. Test, verifica e manutenzione

npm run check controlla la sintassi dei file JavaScript di backend e frontend con node --check. npm test esegue le suite node:test in backend/test. I file di test coprono dashboard admin, allegati, preferiti, presenza amici, token voce LiveKit, cancellazione messaggi, chiamate private, conversazioni, ruoli, sicurezza e scoperta dei server. I test automatici verificano le regole del codice, ma non sostituiscono una prova del browser con database, microfono, webcam e servizio LiveKit reali.

Per verificare una modifica funzionale, seguire un percorso end-to-end: preparare PostgreSQL, applicare migrazioni, avviare backend, creare o usare due account, verificare la rotta interessata e osservare l'evento Socket.IO nell'altra sessione. Per la voce, provare anteprima, permessi dispositivo, ingresso, tile remota, mute, camera, condivisione e uscita su due browser. Per l'interfaccia, controllare i tre temi, il ricaricamento, la larghezza mobile e l'accesso tramite tastiera.

Quando si aggiunge un campo al database, aggiornare schema e migrazione, poi controller, selezioni Prisma, API e rendering. Quando si aggiunge una rotta, applicare autenticazione e autorizzazione coerenti, gestire errori e scrivere un test per una regola sensibile. Quando si aggiunge un controllo UI, aggiornare index.html, dom.js, eventuale controller, events.js e i CSS pertinenti. Per un nuovo evento realtime, documentare payload, stanza destinataria, verifica dei permessi e comportamento in caso di disconnessione.

22. Limiti attuali e glossario

La configurazione LiveKit è obbligatoria per le stanze vocali di gruppo; l'assenza delle variabili produce 503. Il client WebRTC privato potrebbe richiedere TURN in reti restrittive. Il tema è persistente solo nel browser corrente. I campi TOTP presenti nello schema non corrispondono a un percorso di impostazione MFA completo nelle rotte correnti. Giochi, presenza, rate limit e registro delle chiamate attive dipendono da memoria di processo; la scalabilità multiistanza richiede un disegno aggiuntivo. Il controllo /api/health è un'indicazione del processo HTTP, non una verifica profonda di PostgreSQL o LiveKit.

Glossario: API è l'interfaccia HTTP tra browser e backend; Prisma è il livello che traduce operazioni JavaScript in query al database; Socket.IO è il canale realtime per notifiche e signaling; WebRTC trasporta audio e video delle chiamate private; LiveKit gestisce la stanza media di gruppo; STUN/TURN aiutano i peer WebRTC a connettersi attraverso reti diverse; room è una stanza logica Socket.IO o LiveKit, a seconda del contesto; tombstone è un messaggio conservato come record di cancellazione senza contenuto; token di sessione prova l'identità presso il backend; token LiveKit autorizza temporaneamente l'accesso alla stanza vocale.

Appendice A. File da aprire per orientarsi rapidamente

Ingresso browser: frontend/index.html, frontend/assets/js/app.js, frontend/assets/js/modules/startup.js, frontend/assets/js/modules/events.js. Stato e utilità: state.js, api.js, dom.js, ui.js, context.js. Navigazione e UI: controller shell.js, shellLayout.js, navigation.js, channels.js, serverLists.js, friends.js, messageViews.js, settings.js, admin.js. Comunicazione: socket.js, calls.js, voiceRtc.js, voicePanel.js, gameLauncher.js, gameBoard.js.

Ingresso backend: backend/src/main.js, backend/src/app.js, backend/src/routes/index.js, backend/src/config/env.js. Dati e permessi: backend/prisma/schema.prisma, backend/src/lib/prisma.js, backend/src/lib/authorization.js, backend/src/middleware/authMiddleware.js. Flussi: cartelle backend/src/controllers, backend/src/sockets e backend/src/utils. Verifica: backend/test e gli script di backend/package.json.

Appendice B. Regole pratiche per chi estende il progetto

1. Conservare le credenziali esclusivamente nell'ambiente server, senza copiarle in file statici o documenti condivisi.

2. Verificare i permessi sul server per ogni nuova rotta ed evento socket; la UI può solo rendere più chiara la disponibilità delle azioni.

3. Usare transazioni Prisma per operazioni che devono riuscire o fallire insieme, come ingresso tramite invito e aggiornamento di relazioni collegate.

4. Usare textContent o nodi DOM sicuri per dati utente e mantenere i limiti sugli allegati anche lato backend.

5. Distinguere stato persistente nel database da stato volatile di browser e processo quando si progetta il riavvio o lo scaling.

6. Provare le modifiche su desktop, mobile, tastiera, tre temi e condizioni di errore, soprattutto per media e permessi del browser.

Appendice C. Indice analitico dei moduli frontend

frontend/assets/js/app.js è il punto di composizione: l'ordine delle registrazioni rende disponibili le funzioni chiamate durante app.start(). modules/context.js crea l'oggetto condiviso con API, stato, elementi e utility. modules/startup.js collega l'avvio al bootstrap. modules/state.js inizializza stato volatile e legge token e link pendenti. modules/api.js uniforma fetch e le risposte d'errore.

modules/dom.js è l'indice degli ID HTML. Se un elemento viene rinominato, cambiare qui e nel template è il primo passo. modules/events.js collega azioni utente ed effetti, evitando che i controller decidano autonomamente quando un form deve essere inviato. modules/ui.js gestisce elementi ricorrenti: avatar con iniziali, nickname, badge, contenuti dei messaggi, conferme e notifiche brevi. modules/attachments.js dichiara dimensione massima e categorie di allegato riconosciute lato client. Il backend resta la fonte autorevole dei limiti effettivi.

controllers/core.js risolve server, canale e utente privato attualmente selezionati e interpreta link d'invito. controllers/bootstrapData.js popola l'app dopo l'autenticazione. controllers/shell.js cambia schermata principale, aggiorna il dock con nome e ruolo, gestisce mute/deafen e mostra la Home. controllers/shellLayout.js orchestra struttura della shell: rail, sidebar, drawer, resize, menu contestuali e conversazioni recenti. controllers/navigation.js applica le transizioni tra server, canale, chat privata e Home, liberando o aggiornando le viste dipendenti.

controllers/serverLists.js visualizza server di appartenenza, server scopribili e richieste di accesso; aggiorna la selezione dopo un ingresso. controllers/channels.js rende l'elenco dei canali e coordina rinomina o eliminazione autorizzata tramite menu. controllers/friends.js visualizza amici e richieste, ricerca username e invia le azioni corrispondenti alle API. controllers/globalSearch.js gestisce dialogo, query, ritardo di digitazione e paginazione. controllers/settings.js aggiorna profilo, tema, avatar e liste di blocco. controllers/admin.js rende le due viste amministrative usando dati e capability dell'API.

controllers/messageViews.js visualizza messaggi di canale o privati, preferiti, cancellazioni e cronologia delle chiamate. controllers/messageAttachments.js prepara il file, mostra anteprima, invia multipart e pulisce gli URL temporanei. controllers/socket.js ascolta messaggi, presenza, inviti e aggiornamenti del gioco o della chiamata, traducendoli in modifiche dello stato/UI. controllers/calls.js è il client WebRTC a due partecipanti. controllers/voicePanel.js gestisce la visibilità del pannello voce; controllers/voiceRtc.js gestisce anteprima, collegamento LiveKit, tile e controlli dei media.

controllers/gameLauncher.js crea l'invito o il link e seleziona un avversario; controllers/gameBoard.js costruisce l'interfaccia di tris e impiccato. modules/gameButtonControllers.js realizza la piccola animazione dei controller nella barra principale. Ogni file ha un compito riconoscibile, ma l'oggetto app è condiviso: cambiare il nome di un metodo esportato da un controller richiede di cercare tutte le chiamate che lo referenziano negli altri moduli.

Appendice D. Indice analitico dei moduli backend

src/config/env.js carica e valida configurazione. src/lib/prisma.js espone un client Prisma comune. src/lib/authorization.js contiene le regole centrali di ruolo e ambito server. src/lib/adminScope.js seleziona il perimetro delle viste amministrative; src/lib/roleManagement.js contiene logica condivisa per la gestione dei ruoli. src/app.js costruisce Express; src/main.js lo avvia con Socket.IO; src/serverShutdown.js gestisce il termine del processo.

src/routes/index.js raccoglie i router. authRoutes.js espone accesso e profilo; universityRoutes.js la lista e la gestione delle università; serverRoutes.js l'appartenenza e i canali; messageRoutes.js i messaggi di canale; privateRoutes.js messaggi, conversazioni e chiamate private; friendRoutes.js amicizie e blocchi; favoriteRoutes.js i salvati; searchRoutes.js la ricerca; uploadRoutes.js il download protetto; rtcRoutes.js configurazione ICE e token voce; adminRoutes.js operazioni privilegiate. Il router indica con chiarezza metodo HTTP, ordine dei middleware e controller invocato.

src/controllers/authControllers.js valida credenziali e gestisce sessioni. universityControllers.js gestisce atenei. serverControllers.js implementa creazione, inviti, scoperta, richieste, canali e uscita. messageControllers.js centralizza messaggi e regola dei contatti privati. friendControllers.js gestisce amicizie e blocchi. favoriteControllers.js controlla accesso e modifica dei preferiti. searchControllers.js esegue ricerche per ambito. roleControllers.js cambia ruolo, sospensione e admin locale. adminControllers.js fornisce viste aggregate e operazioni su risorse.

src/middleware/authMiddleware.js verifica Bearer, sessione e sospensione; authRateLimit.js limita accessi/registrazioni; adminMiddleware.js esporta il controllo globale; adminAudit.js registra l'esito; errorHandler.js e notFoundHandler.js trasformano gli errori in risposte HTTP coerenti. Un nuovo endpoint sensibile dovrebbe scegliere esplicitamente quali di questi middleware applicare e in quale ordine.

src/sockets/index.js imposta autenticazione e gestori sulla connessione. shared.js trova l'utente e costruisce nomi stanza; authorize.js applica controlli a eventi autenticati; chatHandlers.js scrive e pubblica messaggi testuali; voiceHandlers.js tiene lo stato logico dei canali vocali; callHandlers.js coordina chiamate private e segnali WebRTC; gameHandlers.js conserva e aggiorna le partite; presence.js informa gli amici sui cambiamenti di connessione.

src/utils/sessionTokens.js genera e hash-a i token. serverInvites.js interpreta e riscatta inviti. uploadParser.js riceve il multipart iniziale. messageUpload.js completa la preparazione degli allegati dei messaggi. attachmentTypes.js identifica il tipo dal contenuto; attachmentAccess.js controlla se un file è scaricabile; privateAttachments.js gestisce storage e orfani; messageDeletion.js applica tombstone e pulizia dei file; disconnectUserSockets.js forza la chiusura delle sessioni realtime dopo azioni che cambiano accesso.

La cartella prisma/migrations non è codice applicativo eseguito a ogni richiesta: contiene istruzioni SQL versionate per trasformare il database. backend/test invece è codice eseguibile di verifica. Separare questi tre livelli — schema/migrazioni, applicazione, test — è essenziale quando si corregge un comportamento: una risposta errata può derivare da dati o schema non migrati, da logica server oppure soltanto da rendering client.

Fine della guida.